

Содержание

1. Как пользоваться этим документом	2
2. Файл <code>config.py</code> — конфигурация эксперимента	2
3. Файл <code>data.py</code> — загрузка датасетов и <code>DataLoader</code>	3
3.1. Импорты и константы	3
3.2. <code>load_image_dataset</code> — загрузка и преобразования	4
3.3. <code>take_subset</code> — взять случайное подмножество	5
3.4. <code>split_train_val</code> — разбиение train на train + val	5
3.5. <code>make_loaders</code> — <code>DataLoader</code> -ы для трёх частей	5
3.6. <code>plot_random_images</code> — превью	6
4. Файл <code>cnn.py</code> — модель, обучение, оценка	6
4.1. Импорты	6
4.2. <code>get_device</code> — выбор устройства	7
4.3. <code>set_seed</code> — фиксация всех ГСЧ	7
4.4. <code>ConvBlockA</code> — простой свёрточный блок	7
4.5. <code>ConvBlockB</code> — глубокий блок (две свёртки)	8
4.6. <code>ImageCNN</code> — модель целиком	8
4.7. <code>is_valid_config</code> — проверка размеров	9
4.8. <code>make_optimizer</code> — фабрика оптимизаторов	9
4.9. <code>confusion_matrix</code> — матрица ошибок	9
4.10. <code>train_one_epoch</code> — одна эпоха обучения	10
4.11. <code>evaluate</code> — оценка на val/test	10
4.12. <code>train_model</code> — обучение с ранней остановкой	11
5. Файл <code>block1_main.py</code> — перебор архитектур	12
5.1. Сетка гиперпараметров	12
5.2. <code>build_grid</code> — построение сетки	13
5.3. <code>short_name</code> и <code>cfg_pretty</code> — форматирование подписей	13
5.4. <code>run_one</code> — один эксперимент	13
5.5. <code>train_final_on_full</code> — финальное дообучение	14
5.6. <code>print_results_table</code> — сводная таблица в консоли	15
5.7. Графики	15
5.7.1. <code>plot_curves</code> — train/val loss и accuracy	15
5.7.2. <code>plot_confusion</code> — матрица ошибок	15
5.7.3. <code>plot_param_compare</code> — влияние одного параметра	16
5.7.4. <code>plot_grid_summary</code> — общая картина по всем конфигах	16
5.8. <code>main</code> — главная функция	16
6. Файл <code>block2_main.py</code> — оптимизаторы и переобучение	18
6.1. Базовая архитектура и фабрика модели	18
6.2. Список оптимизаторов	18
6.3. <code>study_optimizers</code>	18
6.4. <code>plot_optimizer_curves</code>	19
6.5. <code>run_overfit_demo</code>	19
6.6. <code>plot_overfit_compare</code>	20
6.7. <code>main</code> блока 2	20
7. Шпаргалка: что говорить на защите	21
7.1. «Зачем нужна функция X»	21
7.2. «Почему именно так»	21

1. Как пользоваться этим документом

Документ построчно разбирает все Python-файлы лабораторной 4. Структура для каждого файла такая:

- сначала короткое объяснение, **за что отвечает файл**;
- потом блоки кода и под каждым — комментарии по каждой важной строчке.

Если на защите спросят «зачем здесь эта строчка» — ищи здесь её объяснение. Хорошая стратегия: для каждой функции уметь сказать (1) что она получает на вход, (2) что делает, (3) что возвращает и зачем.

2. Файл `config.py` — конфигурация эксперимента

Зачем нужен: собирает все параметры эксперимента в одном месте. Все значения читаются из переменных окружения вида `SMGM04_*`. Это позволяет менять режим запуска (датасет, размер выборки, эпохи) **не редактируя код**, что особенно удобно при наличии GUI-лаунчера.

```
import os

SEED = 52
```

- `import os` — модуль для чтения переменных окружения.
- `SEED = 52` — фиксированное зерно генератора случайных чисел. Используется **везде**: при разбиении выборок, инициализации весов, перемешивании. Это ключ к воспроизводимости: одинаковый `SEED` даёт одинаковые результаты при повторных запусках.

```
DATASET_NAME = os.environ.get('SMGM04_DATASET', 'MNIST')
DATA_ROOT = os.environ.get('SMGM04_DATA_ROOT', './data')
```

- `os.environ.get(key, default)` читает переменную окружения; если её нет, берётся `default`.
- `DATASET_NAME` — имя датасета (`MNIST` или `FashionMNIST`). По умолчанию — `MNIST`.
- `DATA_ROOT` — куда сохранять/откуда читать загруженные `torchvision`-датасеты.

```
TRAIN_SUBSET = int(os.environ.get('SMGM04_TRAIN_SUBSET', '10000'))
TEST_SUBSET = int(os.environ.get('SMGM04_TEST_SUBSET', '2000'))
VAL_RATIO = float(os.environ.get('SMGM04_VAL_RATIO', '0.2'))
```

- `TRAIN_SUBSET` — сколько объектов брать из `train` для перебора архитектур. Полный `MNIST` содержит 60 000 объектов; чтобы перебрать 70+ конфигураций, мы берём только подвыборку.
- `TEST_SUBSET` — то же самое для `test`.
- `VAL_RATIO = 0.2` — доля `validation` внутри `train`-подвыборки. То есть из 10 000 `train`: 8000 идут на обучение и 2000 — на валидацию.

```
BATCH_SIZE = int(os.environ.get('SMGM04_BATCH_SIZE', '128'))
N_EPOCHS = int(os.environ.get('SMGM04_N_EPOCHS', '20'))
PATIENCE = int(os.environ.get('SMGM04_PATIENCE', '5'))
MIN_DELTA = float(os.environ.get('SMGM04_MIN_DELTA', '1e-4'))
TARGET_ACC = float(os.environ.get('SMGM04_TARGET_ACC', '0.90'))
FINAL_EPOCHS = int(os.environ.get('SMGM04_FINAL_EPOCHS', str(N_EPOCHS)))
```

- `BATCH_SIZE = 128` — размер мини-батча. Один шаг градиента считается по 128 объектам.
- `N_EPOCHS = 20` — максимум эпох в одном эксперименте.
- `PATIENCE = 5` — параметр ранней остановки. Если 5 эпох подряд `val_acc` не улучшилась — выходим.

- `MIN_DELTA = 1e-4` — какой прирост считать значимым; меньшие колебания не сбрасывают счётчик `bad`.
- `TARGET_ACC = 0.90` — целевая точность. Используется для отчёта «за сколько эпох конфигурация достигла 90%».
- `FINAL_EPOCHS` — сколько эпох гонять финальное дообучение лучшей модели на полном `train`. По умолчанию равно `N_EPOCHS`.

```
NORMALIZE_IMAGES = os.environ.get('SMGM04_NORMALIZE_IMAGES', '1') == '1'
NUM_WORKERS = int(os.environ.get('SMGM04_NUM_WORKERS', '2'))
_pin_memory_raw = os.environ.get('SMGM04_PIN_MEMORY', 'auto').strip().lower()
PIN_MEMORY = None if _pin_memory_raw == 'auto' else _pin_memory_raw in ('1', 'true',
'yes', 'on')
```

- `NORMALIZE_IMAGES` — включать ли вычитание среднего и деление на `std`. Сравнение со строкой `'1'` даёт `bool`.
- `NUM_WORKERS = 2` — сколько отдельных процессов параллельно загружают батчи в `DataLoader`.
- `PIN_MEMORY` — режим закреплённой памяти для ускорения передачи на GPU. Поддерживает три значения: `auto` (по умолчанию — автоопределение), явное `True/False`. На CPU всегда выключается.

```
OVERFIT_TRAIN = int(os.environ.get('SMGM04_OVERFIT_TRAIN', '300'))
OVERFIT_EPOCHS = int(os.environ.get('SMGM04_OVERFIT_EPOCHS', '120'))
```

`DPI = 130`

- `OVERFIT_TRAIN = 300` — размер маленькой подвыборки для эксперимента «специально вызовем переобучение». 300 объектов делятся в пропорции `VAL_RATIO`: 240 `train` + 60 `val`.
- `OVERFIT_EPOCHS = 120` — сколько эпох гонять этот эксперимент **без ранней остановки**, чтобы переобучение успело развернуться.
- `DPI = 130` — разрешение для `matplotlib`-графиков.

3. Файл `data.py` — загрузка датасетов и `DataLoader`

Зачем нужен: загружает MNIST или FashionMNIST из `torchvision`, нормализует, разбивает `train` на `train+val`, оборачивает в `DataLoader`, и умеет рисовать превью.

3.1. Импорты и константы

```
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

from config import SEED, NORMALIZE_IMAGES, NUM_WORKERS, PIN_MEMORY
```

- `numpy` — генератор случайных чисел и массивы индексов.
- `matplotlib` — рисование примеров.
- `Subset` — класс PyTorch, дающий «вид» (`view`) на датасет по списку индексов: новый объект, но без копирования данных.
- `DataLoader` — объект, который батчирует датасет и подгружает данные параллельно.
- `datasets` и `transforms` из `torchvision` — готовые загрузчики MNIST/FashionMNIST и преобразования изображений.

```

DATASET_INFO = {
    'MNIST': {
        'loader': datasets.MNIST,
        'classes': [str(i) for i in range(10)],
        'in_channels': 1,
        'image_size': 28,
    },
    'FashionMNIST': {
        'loader': datasets.FashionMNIST,
        'classes': [
            'T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
            'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Boot',
        ],
        'in_channels': 1,
        'image_size': 28,
    },
}

```

Словарь, по которому код единым образом обрабатывает оба датасета:

- `loader` — класс для загрузки (вызывается ниже как функция).
- `classes` — человекочитаемые имена 10 классов (нужны для confusion matrix и подписей).
- `in_channels = 1` — картинки чёрно-белые, 1 канал.
- `image_size = 28` — сторона картинки.

```

DATASET_NORMALIZATION = {
    'MNIST': {'mean': (0.1307,), 'std': (0.3081,)},
    'FashionMNIST': {'mean': (0.2860,), 'std': (0.3530,)},
}

```

Стандартные средние и стандартные отклонения, **вычисленные по train-выборке** каждого датасета. Это общеизвестные константы.

3.2. `load_image_dataset` — загрузка и преобразования

```

def load_image_dataset(name, data_root):
    if name not in DATASET_INFO:
        raise ValueError(f'Неизвестный датасет: {name}')
    info = DATASET_INFO[name]
    tfms = [transforms.ToTensor()]
    if NORMALIZE_IMAGES and name in DATASET_NORMALIZATION:
        stats = DATASET_NORMALIZATION[name]
        tfms.append(transforms.Normalize(stats['mean'], stats['std']))
    transform = transforms.Compose(tfms)
    train = info['loader'](root=data_root, train=True,
                           download=True, transform=transform)
    test = info['loader'](root=data_root, train=False,
                           download=True, transform=transform)
    return train, test, info

```

- `if name not in DATASET_INFO` — защита от опечатки.
- `tfms = [transforms.ToTensor()]` — `ToTensor`: PIL → `torch.Tensor` формы (1, 28, 28) со значениями в $[0, 1]$.
- `tfms.append(transforms.Normalize(...))` — если `NORMALIZE_IMAGES=True`, добавляем нормализацию: $x' = \frac{x - \mu}{\sigma}$.
- `transforms.Compose(tfms)` — последовательное применение.

- `info['loader'](root=..., train=True, download=True, transform=transform)` — скачивает (если ещё нет) и создаёт обучающую часть.
- `train=True/False` — торчвижн сам понимает, какая часть нужна.
- Возврат: `(train, test, info)` — два датасета и метайнформация.

3.3. `take_subset` — взять случайное подмножество

```
def take_subset(dataset, subset_size, seed=SEED):
    rng = np.random.default_rng(seed)
    n = min(subset_size, len(dataset))
    idx = rng.choice(len(dataset), size=n, replace=False)
    return Subset(dataset, idx.tolist())
```

- `rng = np.random.default_rng(seed)` — **локальный** ГСЧ. Не трогает глобальное состояние NumPy. Один и тот же `seed` всегда даёт те же индексы.
- `n = min(subset_size, len(dataset))` — защита, если попросили больше, чем есть.
- `rng.choice(... replace=False)` — выбор без повторов: индексы уникальны.
- `Subset(dataset, idx.tolist())` — лёгкая обёртка, не копирующая данные.

3.4. `split_train_val` — разбиение train на train + val

```
def split_train_val(dataset, val_ratio, seed=SEED):
    rng = np.random.default_rng(seed)
    idx = np.arange(len(dataset))
    rng.shuffle(idx)
    n_val = int(len(dataset) * val_ratio)
    n_train = len(dataset) - n_val
    return (Subset(dataset, idx[:n_train].tolist()),
            Subset(dataset, idx[n_train:].tolist()))
```

- `np.arange(len(dataset))` — массив `[0, 1, 2, ..., n-1]`.
- `rng.shuffle(idx)` — перемешиваем индексы in-place.
- `n_val = int(... * val_ratio)` — целое число объектов для val (например, 20%).
- Возврат: два `Subset` — сначала train, потом val. Поскольку индексы **перемешаны**, классы в обеих частях распределены равномерно.

3.5. `make_loaders` — DataLoader-ы для трёх частей

```
def make_loaders(train_ds, val_ds, test_ds, batch_size,
                 num_workers=NUM_WORKERS, pin_memory=PIN_MEMORY):
    use_pin_memory = torch.cuda.is_available() if pin_memory is None else (pin_memory and
    torch.cuda.is_available())
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                              num_workers=num_workers, pin_memory=use_pin_memory)
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                            num_workers=num_workers, pin_memory=use_pin_memory)
    test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False,
                             num_workers=num_workers, pin_memory=use_pin_memory)
    return train_loader, val_loader, test_loader
```

- `use_pin_memory` — `pin_memory` имеет смысл только при наличии CUDA. Если в конфиге `auto`, включаем при наличии GPU.
- `shuffle=True` **только для train** — порядок батчей в каждой эпохе разный, что улучшает обучение.
- `shuffle=False` для val/test — там нужен стабильный детерминированный обход.

- `num_workers` — параллельная загрузка с диска.
- `pin_memory=True` — ускоряет передачу на GPU.

3.6. `plot_random_images` — превью

```
def plot_random_images(dataset, class_names, title,
                       rows=4, cols=4, seed=SEED):
    rng = np.random.default_rng(seed)
    fig, axes = plt.subplots(rows, cols, figsize=(2.2 * cols, 2.2 * rows))
    fig.suptitle(title, fontsize=13)
    for ax in axes.ravel():
        i = int(rng.integers(0, len(dataset)))
        img, label = dataset[i]
        ax.imshow(img.squeeze(0), cmap='gray')
        ax.set_title(class_names[int(label)], fontsize=9)
        ax.axis('off')
    plt.tight_layout()
    plt.show()
    plt.close()
```

- `plt.subplots(rows, cols, ...)` — сетка осей.
- `axes.ravel()` — разворачивает 2D-сетку осей в 1D для удобной итерации.
- `rng.integers(0, len(dataset))` — случайный индекс.
- `dataset[i]` — возвращает кортеж `(tensor, label)`.
- `img.squeeze(0)` — убирает размерность канала: $(1, 28, 28) \rightarrow (28, 28)$.
- `cmap='gray'` — серое изображение.
- `class_names[int(label)]` — подпись класса.
- `ax.axis('off')` — скрыть оси.
- `plt.show() + plt.close()` — показать и освободить память.

4. Файл `сnn.py` — модель, обучение, оценка

Зачем нужен: содержит саму CNN-архитектуру, фабрику оптимизаторов и функции `train_one_epoch`, `evaluate`, `train_model` — ядро лабораторной.

4.1. Импорты

```
import copy
import math
import random
import time

import numpy as np
import torch
import torch.nn as nn

from config import SEED
```

- `copy` — глубокое копирование `state_dict()` модели для сохранения лучшей версии.
- `math.floor` — для проверки `is_valid_config`.
- `random`, `numpy`, `torch` — три источника случайности, которые синхронно сидируем.
- `time.perf_counter()` — замер длительности обучения.
- `torch.nn as nn` — слои сети (`Linear`, `Conv2d`, `ReLU`, ...).

4.2. `get_device` — выбор устройства

```
def get_device():
    if torch.cuda.is_available():
        return torch.device('cuda')
    if torch.backends.mps.is_available() and torch.backends.mps.is_built():
        return torch.device('mps')
    return torch.device('cpu')
```

Иерархия выбора: NVIDIA CUDA → Apple Metal (MPS) → CPU. `torch.backends.mps.is_built()` нужен потому, что бывают сборки PyTorch без поддержки MPS даже на macOS.

4.3. `set_seed` — фиксация всех ГСЧ

```
def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
```

Сидируем **три библиотеки** (Python `random`, NumPy, PyTorch) и плюс CUDA-устройства. Без этого инициализация весов и порядок батчей плавали бы между запусками, и сравнение архитектур стало бы нечестным.

4.4. `ConvBlockA` — простой свёрточный блок

```
class ConvBlockA(nn.Module):
    def __init__(self, in_ch, out_ch, conv_k, conv_s, pool_k, pool_s):
        super().__init__()
        pad = conv_k // 2
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=conv_k,
                      stride=conv_s, padding=pad),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=pool_k, stride=pool_s),
        )

    def forward(self, x):
        return self.block(x)
```

- `nn.Module` — базовый класс всех нейросетевых модулей PyTorch.
- `super().__init__()` — обязательный вызов конструктора `nn.Module`, без него не работает регистрация параметров.
- `pad = conv_k // 2` — такой padding при `stride=1` сохраняет пространственный размер (например, $3 \rightarrow 1$, $5 \rightarrow 2$, $7 \rightarrow 3$).
- `nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding)` — свёрточный слой: `in_ch` входных каналов, `out_ch` выходных, ядро `kernel_size`.
- `nn.ReLU()` — нелинейность.
- `nn.MaxPool2d(...)` — max-pooling: уменьшает пространственный размер.
- `nn.Sequential(...)` — контейнер, прогоняющий вход через слои по порядку.
- `forward(x)` — метод, описывающий прямой проход. Вызывается автоматически при `model(x)`.

4.5. ConvBlockB — глубокий блок (две свёртки)

```
class ConvBlockB(nn.Module):
    def __init__(self, in_ch, out_ch, conv_k, conv_s, pool_k, pool_s):
        super().__init__()
        pad = conv_k // 2
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=conv_k,
                      stride=conv_s, padding=pad),
            nn.ReLU(),
            nn.Conv2d(out_ch, out_ch, kernel_size=conv_k,
                      stride=1, padding=pad),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=pool_k, stride=pool_s),
        )
```

Отличие от A: добавлены **вторая Conv2d + ReLU перед pooling**. Вторая свёртка имеет **stride=1** (размер уже не уменьшаем) и работает по **out_ch**. Это даёт более выразительный блок ценой большего числа параметров.

4.6. ImageCNN — модель целиком

```
class ImageCNN(nn.Module):
    def __init__(self, in_channels, num_classes, block_type, channels,
                  conv_k, conv_s, pool_k, pool_s, dropout_rate=0.0):
        super().__init__()
        if block_type not in ('a', 'b'):
            raise ValueError("block_type должен быть 'a' или 'b'")
        Block = ConvBlockA if block_type == 'a' else ConvBlockB
```

- **dropout_rate=0.0** по умолчанию — регуляризация выключена.
- **Block = ConvBlockA if ... else ConvBlockB** — **выбор класса** (не объекта). Дальше используем как фабрику.

```
        layers = []
        prev = in_channels
        for ch in channels:
            layers.append(Block(prev, ch, conv_k, conv_s, pool_k, pool_s))
            prev = ch
        self.features = nn.Sequential(*layers)
```

Строим стек блоков. На каждом шаге **prev** — число каналов, которое выдаёт предыдущий блок (вход для следующего). Если **channels=(32, 64)**, то получаем два блока: $1 \rightarrow 32$ и $32 \rightarrow 64$.

```
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Dropout(dropout_rate),
            nn.Linear(prev, num_classes),
        )
```

- **AdaptiveAvgPool2d((1, 1))** — глобальный average pooling: каждая карта признаков становится одним числом. **Не нужно вручную считать размер тензора перед Linear**.
- **Flatten()** — $(B, C, 1, 1) \rightarrow (B, C)$.
- **Dropout(p)** — во время обучения случайно зануляет долю p компонент. На eval отключается автоматически.

- `Linear(prev, num_classes)` — финальный классификатор: 10 логитов.

```
def forward(self, x):
    return self.classifier(self.features(x))
```

Прямой проход — сначала свёртки, потом классификатор.

4.7. `is_valid_config` — проверка размеров

```
def is_valid_config(image_size, num_blocks, conv_k, conv_s, pool_k, pool_s):
    s = image_size
    pad = conv_k // 2
    for _ in range(num_blocks):
        s = math.floor((s + 2 * pad - conv_k) / conv_s + 1)
        if s < pool_k:
            return False
        s = math.floor((s - pool_k) / pool_s + 1)
        if s < 1:
            return False
    return True
```

Эмулирует расчёт пространственного размера тензора:

- формула свёртки: $\text{out} = \left\lfloor \frac{\text{in} + 2p - k}{s} + 1 \right\rfloor$;
- формула pooling (без padding): $\text{out} = \left\lfloor \frac{\text{in} - k}{s} + 1 \right\rfloor$.

Если на каком-то шаге размер становится меньше ядра pooling или меньше 1 — конфигурация невалидна и её сразу отбрасываем (иначе модель упадёт при прямом проходе).

4.8. `make_optimizer` — фабрика оптимизаторов

```
def make_optimizer(name, params, lr):
    if name == 'Adam':
        return torch.optim.Adam(params, lr=lr)
    if name == 'AdamW':
        return torch.optim.AdamW(params, lr=lr, weight_decay=1e-4)
    if name == 'SGD':
        return torch.optim.SGD(params, lr=lr, momentum=0.9, nesterov=True)
    raise ValueError(f'Неизвестный оптимизатор: {name}')
```

- `Adam` — адаптивный, чаще всего «дефолтный выбор».
- `AdamW` — Adam с decoupled weight decay (правильнее, чем `Adam(weight_decay=...)`); `1e-4` — стандартный выбор.
- `SGD` с `momentum=0.9` и `nesterov=True` — классический оптимизатор, требует более тщательной настройки `lr`.

4.9. `confusion_matrix` — матрица ошибок

```
def confusion_matrix(y_true, y_pred, n_classes):
    cm = np.zeros((n_classes, n_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[int(t), int(p)] += 1
    return cm
```

- `cm[i, j]` — число объектов истинного класса `i`, отнесённых к классу `j`.
- Диагональ — правильные ответы; недиагональные элементы — ошибки.

4.10. `train_one_epoch` — одна эпоха обучения

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0.0
    total_correct = 0
    total = 0
    for x, y in loader:
        x = x.to(device)
        y = y.to(device)
        optimizer.zero_grad()
        out = model(x)
        loss = criterion(out, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * y.size(0)
        total_correct += (out.argmax(dim=1) == y).sum().item()
        total += y.size(0)
    return total_loss / total, total_correct / total
```

- `model.train()` — режим обучения: Dropout активен, BatchNorm обновляет статистики.
- `for x, y in loader` — проход по всем батчам.
- `x.to(device)` — перенос тензора на CPU/GPU.
- `optimizer.zero_grad()` — **обязательно** обнулить старые градиенты (иначе они накапливаются).
- `out = model(x)` — прямой проход.
- `loss = criterion(out, y)` — скаляр, среднее по батчу.
- `loss.backward()` — автоматическое дифференцирование, заполняет `.grad` для всех параметров.
- `optimizer.step()` — обновление весов по правилу выбранного оптимизатора.
- `loss.item() * y.size(0)` — так как `loss` уже усреднён по батчу, чтобы получить **сумму**, умножаем на размер батча. Затем в конце делим на `total` — получаем точное среднее по эпохе.
- `(out.argmax(dim=1) == y).sum()` — индекс класса с максимальным логитом и сравнение с истинной.

4.11. `evaluate` — оценка на val/test

```
@torch.no_grad()
def evaluate(model, loader, criterion, device, return_preds=False):
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total = 0
    y_true_all, y_pred_all = [], []
    for x, y in loader:
        x = x.to(device)
        y = y.to(device)
        out = model(x)
        loss = criterion(out, y)
        preds = out.argmax(dim=1)
        total_loss += loss.item() * y.size(0)
        total_correct += (preds == y).sum().item()
        total += y.size(0)
    if return_preds:
        y_true_all.extend(y.cpu().numpy())
        y_pred_all.extend(preds.cpu().numpy())
    avg_loss = total_loss / total
```

```

avg_acc = total_correct / total
if return_preds:
    return avg_loss, avg_acc, np.array(y_true_all), np.array(y_pred_all)
return avg_loss, avg_acc

```

- `@torch.no_grad()` — отключает построение графа autograd. Экономит память и ускоряет вычисления; на eval градиенты не нужны.
- `model.eval()` — выключает Dropout, переключает BatchNorm в режим использования бегущих статистик.
- `if return_preds:` — для confusion matrix нам нужны **все** истинные и предсказанные метки.
- `y.cpu().numpy()` — переносим обратно на CPU (numpy не понимает GPU-тензоры).
- В конце возвращаем либо 2 значения (только метрики), либо 4 (плюс метки для CM).

4.12. `train_model` — обучение с ранней остановкой

```

def train_model(model, train_loader, val_loader, optimizer, criterion, device,
                n_epochs, target_acc, patience, min_delta,
                log_every=5, tag=''):
    history = {'train_loss': [], 'val_loss': [],
               'train_acc': [], 'val_acc': []}
    best_val_acc = -1.0
    best_state = copy.deepcopy(model.state_dict())
    best_epoch = 0
    epoch_to_target = None
    bad = 0
    t0 = time.perf_counter()

```

- `history` — 4 списка с метриками по эпохам, нужны для графиков.
- `best_val_acc = -1.0` — любое реальное значение accuracy будет больше.
- `best_state = copy.deepcopy(model.state_dict())` — сохраняем стартовое состояние (страховка, если ни одна эпоха не улучшит).
- `bad` — счётчик «эпох без улучшения», для ранней остановки.
- `t0` — старт замера времени.

```

for epoch in range(1, n_epochs + 1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader,
                                      criterion, optimizer, device)
    val_loss, val_acc = evaluate(model, val_loader, criterion, device)

    history['train_loss'].append(tr_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(tr_acc)
    history['val_acc'].append(val_acc)

```

Цикл по эпохам: одна обучающая итерация + оценка на val + сохранение метрик.

```

if epoch == 1 or epoch % log_every == 0 or epoch == n_epochs:
    prefix = f'[{tag}] ' if tag else ''
    print(f' {prefix}эпоха {epoch:>3}/{n_epochs} | ...')

```

Логируем не каждую эпоху, а с шагом `log_every` — иначе вывод раздувается на 70+ конфигураций.

```

if epoch_to_target is None and val_acc >= target_acc:
    epoch_to_target = epoch

```

Запоминаем **первую** эпоху, на которой `val_acc >= 0.90`. Дальше не обновляем (отсюда `is None`).

```
if val_acc > best_val_acc + min_delta:
    best_val_acc = val_acc
    best_epoch = epoch
    best_state = copy.deepcopy(model.state_dict())
    bad = 0
else:
    bad += 1
if bad >= patience:
    print(f' -> ранняя остановка на эпохе {epoch}')
    break
```

Сердце ранней остановки:

- если улучшение больше `min_delta` — сохраняем веса и сбрасываем счётчик;
- иначе наращиваем `bad`;
- при `bad >= patience` выходим из цикла.

```
model.load_state_dict(best_state)
duration = time.perf_counter() - t0
return {
    'history': history,
    'best_val_acc': best_val_acc,
    'best_epoch': best_epoch,
    'epoch_to_target': epoch_to_target,
    'duration_sec': duration,
    'trained_epochs': len(history['train_loss']),
}
```

После цикла **возвращаемся к лучшему состоянию**. Это критично: иначе модель осталась бы с весами последней (часто — худшей) эпохи. Возвращаем словарь со всеми метриками для дальнейшей таблицы и графиков.

5. Файл `block1_main.py` — перебор архитектур

Зачем нужен: строит сетку конфигураций, обучает каждую, выбирает лучшую по `val_acc`, переобучает на полном `train`, рисует графики.

5.1. Сетка гиперпараметров

```
BLOCK_TYPES = ['a', 'b']
CHANNELS_OPTIONS = [(16, 32), (32, 64)]
CONV_KERNELS = [3, 5]
CONV_STRIDES = [1]
POOL_KERNELS = [2]
POOL_STRIDES = [1, 2]
LEARNING_RATES = [1e-3]
```

Что варьируется и зачем:

- `BLOCK_TYPES` — простой блок vs глубокий;
- `CHANNELS_OPTIONS` — ширина модели;
- `CONV_KERNELS` — размер receptive field;
- `POOL_STRIDES` — агрессивность уменьшения пространства.

Параметры с одним значением (`CONV_STRIDES`, `POOL_KERNELS`, `LEARNING_RATES`) тоже хранятся как списки — единообразный формат для `itertools.product`.

5.2. `build_grid` — построение сетки

```
def build_grid(image_size=28):
    grid = []
    config_id = 1
    for block_type, channels, conv_k, conv_s, pool_k, pool_s, lr in itertools.product(
        BLOCK_TYPES, CHANNELS_OPTIONS, CONV_KERNELS, CONV_STRIDES,
        POOL_KERNELS, POOL_STRIDES, LEARNING_RATES,
    ):
        if not is_valid_config(image_size, len(channels), conv_k, conv_s,
                               pool_k, pool_s):
            continue
        grid.append({
            'id': config_id,
            'block_type': block_type, 'channels': channels,
            'conv_k': conv_k, 'conv_s': conv_s,
            'pool_k': pool_k, 'pool_s': pool_s,
            'lr': lr,
        })
        config_id += 1
    return grid
```

- `itertools.product(...)` — декартово произведение всех списков.
- `is_valid_config(...)` — отбрасываем заведомо неработающие конфигурации (например, ядро 5 при `pool_stride=2` через 3 блока даст размер 0).
- `config_id` — последовательный номер для отчётов и графиков.

5.3. `short_name` и `cfg_pretty` — форматирование подписей

```
def short_name(cfg):
    return (f"#{cfg['id']:02d} blk{cfg['block_type']} "
            f"ch={cfg['channels']} ck={cfg['conv_k']} ps={cfg['pool_s']}")

def cfg_pretty(cfg):
    return (f"block={cfg['block_type']} | channels={cfg['channels']} | "
            f"conv(k={cfg['conv_k']}, s={cfg['conv_s']}) | "
            f"pool(k={cfg['pool_k']}, s={cfg['pool_s']}) | "
            f"lr={cfg['lr']}")
```

- `short_name` — короткая подпись для тиков графика.
- `cfg_pretty` — подробная строка для логов.

5.4. `run_one` — один эксперимент

```
def run_one(cfg, train_loader, val_loader, info, device):
    set_seed(SEED)
    model = ImageCNN(
        in_channels=info['in_channels'],
        num_classes=len(info['classes']),
        block_type=cfg['block_type'], channels=cfg['channels'],
        conv_k=cfg['conv_k'], conv_s=cfg['conv_s'],
        pool_k=cfg['pool_k'], pool_s=cfg['pool_s'],
    ).to(device)
```

```

optimizer = make_optimizer('Adam', model.parameters(), cfg['lr'])
criterion = nn.CrossEntropyLoss()
n_params = sum(p.numel() for p in model.parameters())
print(f"\n>>> {cfg_pretty(cfg)} | params={n_params:,}")
res = train_model(model, train_loader, val_loader, optimizer, criterion,
                  device, n_epochs=N_EPOCHS, target_acc=TARGET_ACC,
                  patience=PATIENCE, min_delta=MIN_DELTA, log_every=2,
                  tag=f"#{cfg['id']:02d}")

res['cfg'] = cfg
res['model'] = model
res['n_params'] = n_params
return res

```

- `set_seed(SEED)` **внутри** — **перед каждой** конфигурацией. Так инициализация весов для всех конфигураций одинаково «случайна» — сравнение честное.
- `.to(device)` — модель на нужное устройство.
- `nn.CrossEntropyLoss()` — loss для классификации (внутри делает log-softmax + NLL).
- `sum(p.numel() for p in model.parameters())` — общее число обучаемых параметров.
- `train_model(...)` — собственно обучение.
- В конце дополняем результат самой `cfg`, моделью и числом параметров.

5.5. `train_final_on_full` — финальное дообучение

```

def train_final_on_full(best_cfg, train_full_ds, test_ds, info, device,
                       n_epochs=FINAL_EPOCHS):
    set_seed(SEED)
    model = ImageCNN(...).to(device)
    optimizer = make_optimizer('Adam', model.parameters(), best_cfg['lr'])
    criterion = nn.CrossEntropyLoss()

```

После выбора лучшей архитектуры обучаем её **с нуля** на полном `train_full_ds` (60k объектов). Это разумно, потому что в продакшен идёт модель, обученная на максимуме доступных данных.

```

train_loader_full = DataLoader(train_full_ds, batch_size=BATCH_SIZE,
                               shuffle=True, num_workers=NUM_WORKERS,
                               pin_memory=use_pin_memory)
test_loader_full = DataLoader(test_ds, batch_size=BATCH_SIZE,
                              shuffle=False, num_workers=NUM_WORKERS,
                              pin_memory=use_pin_memory)

```

Свои loader-ы (без val), с тем же `pin_memory`-режимом, что и в основной части.

```

for epoch in range(1, n_epochs + 1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader_full,
                                      criterion, optimizer, device)

    history['train_loss'].append(tr_loss)
    history['train_acc'].append(tr_acc)

```

Простой цикл без ранней остановки — архитектура уже выбрана.

```

test_loss, test_acc, y_true, y_pred = evaluate(
    model, test_loader_full, criterion, device, return_preds=True)

```

Финальная **честная** оценка на test, плюс `y_true/y_pred` для confusion matrix.

5.6. `print_results_table` — сводная таблица в консоли

```
def print_results_table(results):
    print('\n' + '=' * 110)
    print(f"{'#':>3} | {'blk':>3} | ...")
    print('-' * 110)
    for r in sorted(results, key=lambda x: -x['best_val_acc']):
        c = r['cfg']
        ep90 = r['epoch_to_target'] if r['epoch_to_target'] else '-'
        print(f"{'id':>3} | {'block_type':>3} | ...")
    print('=' * 110)
```

- `sorted(..., key=lambda x: -x['best_val_acc'])` — сортировка по убыванию `val_acc` (отрицательный знак = по убыванию).
- `:>3, :>10` — выравнивание по правому краю с заданной шириной.
- `'-'` — если цели 90% не достигли.

5.7. Графики

5.7.1. `plot_curves` — train/val loss и accuracy

```
def plot_curves(history, title, target_acc=TARGET_ACC, best_epoch=None):
    fig, (ax_l, ax_a) = plt.subplots(1, 2, figsize=(13, 4.2))
    ax_l.plot(history['train_loss'], label='train loss')
    ax_l.plot(history['val_loss'], label='val loss')
    if best_epoch is not None:
        ax_l.axvline(best_epoch - 1, color='crimson', ls='--',
                    label='best epoch')
```

- Левый subplot — loss, правый — accuracy.
- `axvline(best_epoch - 1)` — **вертикальная** линия в момент лучшей эпохи (минус 1, потому что эпохи нумеруем с 1, а ось — с 0).

```
ax_a.axhline(target_acc, color='green', ls='--', ...)
```

- `axhline(target_acc)` — **горизонтальная** линия на уровне 0.90, чтобы было видно, когда кривая её пересекает.

5.7.2. `plot_confusion` — матрица ошибок

```
def plot_confusion(cm, class_names, title):
    n = len(class_names)
    fig, ax = plt.subplots(figsize=(8, 7))
    im = ax.imshow(cm, cmap='Blues')
    ax.set_xticklabels(class_names, rotation=45, ha='right')
    ax.set_yticklabels(class_names)
    thr = cm.max() / 2 if cm.size else 0
    for i in range(n):
        for j in range(n):
            color = 'white' if cm[i, j] > thr else 'black'
            ax.text(j, i, str(cm[i, j]), ha='center', va='center',
                    fontsize=8, color=color)
    plt.colorbar(im, ax=ax, fraction=0.046)
```

- `imshow(cm, cmap='Blues')` — рисует матрицу как тепловую карту.
- `rotation=45, ha='right'` — метки классов под углом, чтобы не наезжали друг на друга.

- `thr = cm.max() / 2` — порог: на тёмных ячейках пишем белым текстом, на светлых — чёрным.
- Двойной цикл `i, j` — подписываем число в каждой клетке.

5.7.3. `plot_param_compare` — влияние одного параметра

```
def plot_param_compare(results, param_name, getter, target_acc=TARGET_ACC):
    groups = {}
    for r in results:
        groups.setdefault(getter(r['cfg']), []).append(r['best_val_acc'])
    keys = sorted(groups, key=str)
    means = [float(np.mean(groups[k])) for k in keys]
    stds = [float(np.std(groups[k])) for k in keys]
    ax.bar(x, means, yerr=stds, capsize=4, color='seagreen')
```

- `getter` — функция-извлекатель значения параметра из `cfg` (например, `lambda c: c['block_type']`).
- `groups.setdefault(key, [])` — если ключа ещё нет, кладём пустой список; затем `.append(value)`.
- `means` и `stds` — среднее и `std val_acc` по группам с одинаковым значением параметра.
- `bar(..., yerr=stds, capsize=4)` — столбчатая диаграмма с усами стандартного отклонения.

5.7.4. `plot_grid_summary` — общая картина по всем конфигам

```
def plot_grid_summary(results, target_acc=TARGET_ACC):
    sorted_res = sorted(results, key=lambda r: -r['best_val_acc'])
    labels = [short_name(r['cfg']) for r in sorted_res]
    accs = [r['best_val_acc'] for r in sorted_res]
    eps90 = [r['epoch_to_target'] or 0 for r in sorted_res]
```

Готовим три параллельных списка:

- `labels` — короткие имена конфигов;
- `accs` — их `val_acc`;
- `eps90` — эпоха достижения 90%; `None or 0` даёт 0 (нули рисуются как «не достигли»).

```
fig, axes = plt.subplots(2, 1, figsize=(max(10, len(labels) * 0.55), 8))
```

`max(10, len(labels) * 0.55)` — автоматический выбор ширины: чем больше конфигов, тем шире рисунок.

Дальше две `bar`-диаграммы: верхняя по `accs`, нижняя по `eps90`. Над столбиком в верхней панели рисуется числовая подпись через `axes[0].text(...)`.

5.8. `main` — главная функция

```
def main():
    set_seed(SEED)
    device = get_device()
    train_full, test_full, info = load_image_dataset(DATASET_NAME, DATA_ROOT)

    train_grid = take_subset(train_full, TRAIN_SUBSET, seed=SEED)
    train_ds, val_ds = split_train_val(train_grid, VAL_RATIO, seed=SEED)
    test_ds = take_subset(test_full, TEST_SUBSET, seed=SEED + 1)
```

- Глобальный `seed`.
- Загрузка датасета.
- Подвыборка `train` для перебора, разбиение на `train/val`.

- **Своя** подвыборка test (с другим seed `SEED + 1` — чтобы train и test заведомо не пересекались по индексам).

```
train_loader, val_loader, test_loader = make_loaders(
    train_ds, val_ds, test_ds, batch_size=BATCH_SIZE)
plot_random_images(train_ds, info['classes'],
                    f'{DATASET_NAME}: примеры из train')
grid = build_grid(image_size=info['image_size'])
```

DataLoader-ы, преью датасета, сетка конфигураций.

```
results = []
for cfg in grid:
    results.append(run_one(cfg, train_loader, val_loader, info, device))
print_results_table(results)
best = max(results, key=lambda r: r['best_val_acc'])
```

- Прогоняем всю сетку.
- Выводим таблицу.
- **Лучшая конфигурация = максимум по val_acc** (не по test).

```
test_loss, test_acc, y_true, y_pred = evaluate(
    best['model'], test_loader, nn.CrossEntropyLoss(), device,
    return_preds=True)
final_res = train_final_on_full(best['cfg'], train_full, test_full,
                                info, device)
```

- Оценка лучшей модели на test-подвыборке (сразу с метками для будущей СМ).
- Финальное переобучение на полном train.

```
plot_grid_summary(results)
plot_curves(best['history'], title=short_name(best['cfg']),
            best_epoch=best['best_epoch'])
cm = confusion_matrix(final_res['y_true'], final_res['y_pred'],
                      len(info['classes']))
plot_confusion(cm, info['classes'], ...)
plot_param_compare(results, 'block_type', lambda c: c['block_type'])
plot_param_compare(results, 'channels', lambda c: str(c['channels']))
plot_param_compare(results, 'conv_kernel_size', lambda c: c['conv_k'])
plot_param_compare(results, 'pool_stride', lambda c: c['pool_s'])
```

Все 8 графиков отчёта рисуются здесь:

1. `plot_grid_summary` — картинка `*-2.png`;
2. `plot_curves` — `*-3.png`;
3. `plot_confusion` — `*-4.png`;
4. четыре `plot_param_compare` — `*-5.png` ... `*-8.png`.

```
reached = [r for r in results if r['epoch_to_target'] is not None]
if reached:
    eps = [r['epoch_to_target'] for r in reached]
    print(f' достигли цели: {len(reached)}/{len(results)}')
    print(f' минимум эпох: {min(eps)}')
    print(f' медиана: {int(np.median(eps))}')
```

Финальная статистика: сколько конфигов вообще достигли 90%, минимум и медиана эпох.

```
if __name__ == '__main__':
    main()
```

Запуск только при прямом вызове `python block1_main.py`. При `import` модуля `main()` не вызывается.

6. Файл `block2_main.py` — оптимизаторы и переобучение

Зачем нужен: на уже выбранной хорошей архитектуре исследует две вещи:

1. влияние оптимизатора и `lr` на скорость и качество обучения;
2. переобучение на маленькой подвыборке и роль Dropout.

6.1. Базовая архитектура и фабрика модели

```
BASE_CFG = {
    'block_type': 'b',
    'channels': (32, 64),
    'conv_k': 3, 'conv_s': 1,
    'pool_k': 2, 'pool_s': 2,
}

def make_model(info, device, dropout=0.0):
    return ImageCNN(
        in_channels=info['in_channels'],
        num_classes=len(info['classes']),
        block_type=BASE_CFG['block_type'], channels=BASE_CFG['channels'],
        conv_k=BASE_CFG['conv_k'], conv_s=BASE_CFG['conv_s'],
        pool_k=BASE_CFG['pool_k'], pool_s=BASE_CFG['pool_s'],
        dropout_rate=dropout,
    ).to(device)
```

- `BASE_CFG` — хорошая (но не самая большая) архитектура из топа блока 1: блок **B**, каналы (32, 64), ядро 3.
- `make_model` — единая точка создания модели, чтобы сравнения были честными. Только `dropout` варьируется.

6.2. Список оптимизаторов

```
OPT_VARIANTS = [
    ('Adam', 1e-3),
    ('Adam', 3e-3),
    ('AdamW', 1e-3),
    ('SGD', 1e-2),
    ('SGD', 5e-2),
]
```

5 пар «оптимизатор + `lr`». Включён один и тот же оптимизатор с **разными** `lr` (`Adam@1e-3` vs `Adam@3e-3`, `SGD@1e-2` vs `SGD@5e-2`), чтобы заодно увидеть чувствительность к `lr`.

6.3. `study_optimizers`

```
def study_optimizers(train_loader, val_loader, info, device,
                     n_epochs=10, patience=4):
    results = []
    criterion = nn.CrossEntropyLoss()
```

```

for opt_name, lr in OPT_VARIANTS:
    set_seed(SEED)
    model = make_model(info, device)
    optimizer = make_optimizer(opt_name, model.parameters(), lr)
    res = train_model(model, train_loader, val_loader, optimizer,
                      criterion, device, n_epochs=n_epochs,
                      target_acc=TARGET_ACC, patience=patience,
                      min_delta=1e-4, log_every=1,
                      tag=f'{opt_name}-{lr}')
    res['opt_name'] = opt_name
    res['lr'] = lr
    results.append(res)

```

Контролируемый эксперимент:

- одни и те же данные (общие `train_loader` / `val_loader`);
- одна и та же архитектура (`make_model`);
- одна и та же инициализация (`set_seed(SEED)` перед каждым запуском);
- меняется только оптимизатор и `lr`.

6.4. plot_optimizer_curves

```

def plot_optimizer_curves(results, target_acc=TARGET_ACC):
    fig, ax = plt.subplots(figsize=(9, 5))
    for r in results:
        epochs = np.arange(1, len(r['history']['val_acc']) + 1)
        ax.plot(epochs, r['history']['val_acc'], 'o-',
                label=f"{r['opt_name']} lr={r['lr']}")
        ax.axhline(target_acc, color='red', ls='--', ...)

```

Все пять кривых `val_acc` на одной оси, плюс горизонтальная линия `target=0.90`. Это и есть `*-2-9.png`.

6.5. run_overfit_demo

```

def run_overfit_demo(info, device):
    train_full, test_full, _ = load_image_dataset(DATASET_NAME, DATA_ROOT)
    train_small = take_subset(train_full, OVERFIT_TRAIN, seed=SEED)
    train_ds, val_ds = split_train_val(train_small, VAL_RATIO, seed=SEED)
    test_ds = take_subset(test_full, TEST_SUBSET, seed=SEED + 1)

```

Берём **очень маленькую** train-подвыборку (`OVERFIT_TRAIN = 300` объектов: 240 train + 60 val). На таком объёме модель неизбежно переобучится — это и нужно для демонстрации.

```

print('\n--- без регуляризации (dropout=0) ---')
set_seed(SEED)
model_no = make_model(info, device, dropout=0.0)
optimizer = make_optimizer('Adam', model_no.parameters(), 1e-3)
res_no = train_model(model_no, train_loader, val_loader, optimizer,
                     criterion, device, n_epochs=OVERFIT_EPOCHS,
                     target_acc=1.0, patience=OVERFIT_EPOCHS,
                     min_delta=0.0, log_every=20, tag='no-reg')

```

Важные хитрости:

- `target_acc=1.0` — цель «недостижима», чтобы маркер в логге никогда не сработал;

- `patience=OVERFIT_EPOCHS` — ранняя остановка фактически **выключена**: даём модели свободно переобучиться все 120 эпох;
- `min_delta=0.0` — то же самое, любое улучшение зачтётся.

```
print('\n--- c Dropout=0.4 ---')
set_seed(SEED)
model_dr = make_model(info, device, dropout=0.4)
optimizer = make_optimizer('Adam', model_dr.parameters(), 1e-3)
res_dr = train_model(model_dr, train_loader, val_loader, optimizer,
                     criterion, device, n_epochs=OVERFIT_EPOCHS, ...)
```

Полностью то же самое, кроме `dropout=0.4`. Это и есть **controlled experiment**: единственное отличие — регуляризация.

```
plot_overfit_compare(res_no, res_dr)
for tag, r in [('без per.', res_no), ('dropout 0.4', res_dr)]:
    h = r['history']
    gap = h['train_acc'][-1] - h['val_acc'][-1]
    print(f" {tag:<14} | train_acc=... | val_acc=... | gap={gap:+.4f}")
```

`gap = train_acc - val_acc` — величина переобучения. Без регуляризации `gap` большой и положительный (модель «знает train лучше, чем val»), с Dropout — значительно меньше или даже отрицательный.

6.6. `plot_overfit_compare`

```
def plot_overfit_compare(res_no, res_dr):
    fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
    h1, h2 = res_no['history'], res_dr['history']
    axes[0].plot(h1['train_loss'], label='train (no-reg)')
    axes[0].plot(h1['val_loss'], label='val (no-reg)')
    axes[0].plot(h2['train_loss'], label='train (dropout)', ls='--')
    axes[0].plot(h2['val_loss'], label='val (dropout)', ls='--')
```

Две панели: loss и accuracy. На каждой — **4 кривые**: train/val для обоих режимов. Сплошные линии — без регуляризации, пунктирные — с Dropout. Ровно эта картинка — `*-2-10.png`.

6.7. `main` блока 2

```
def main():
    set_seed(SEED)
    device = get_device()
    train_full, test_full, info = load_image_dataset(DATASET_NAME, DATA_ROOT)
    train_full = take_subset(train_full, TRAIN_SUBSET, seed=SEED)
    train_ds, val_ds = split_train_val(train_full, VAL_RATIO, seed=SEED)
    test_ds = take_subset(test_full, TEST_SUBSET, seed=SEED + 1)
    train_loader, val_loader, _ = make_loaders(
        train_ds, val_ds, test_ds, batch_size=BATCH_SIZE)

    opt_results = study_optimizers(train_loader, val_loader, info, device,
                                   n_epochs=min(N_EPOCHS, 10))
    plot_optimizer_curves(opt_results)
    run_overfit_demo(info, device)
```

- Готовим стандартную train/val-подвыборку (как в блоке 1).
- Запускаем сравнение оптимизаторов на **максимум 10 эпох** (для скорости).

- Рисуем кривые.
- Запускаем демонстрацию переобучения, которая внутри сама готовит свои маленькие подвыборки.

```
if __name__ == '__main__':
    main()
```

Запуск только при прямом вызове.

7. Шпаргалка: что говорить на защите

7.1. «Зачем нужна функция X»

Функция

`set_seed`

Главная мысль

Воспроизводимость: одинаковый seed → одинаковая инициализация весов и порядок батчей.

`get_device`

Прозрачно использовать GPU там, где он есть (CUDA/MPS/CPU).

`load_image_dataset`

Скачать датасет, привести к тензору, нормализовать.

`take_subset`

Взять случайное (но воспроизводимое) подмножество для ускорения.

`split_train_val`

Разбить train на train/val, чтобы val честно оценивал обучение.

`make_loaders`

Завернуть Dataset в DataLoader: батчи + shuffle + параллельная загрузка.

`ConvBlockA/B`

Базовый строительный кирпич CNN; **B** глубже на одну свёртку.

`ImageCNN`

Стек блоков + AdaptiveAvgPool + Dropout + Linear — вся модель.

`is_valid_config`

Защита от конфигураций, у которых пространство схлопывается в 0.

`make_optimizer`

Единая фабрика для Adam/AdamW/SGD.

`train_one_epoch`

Один проход по train: forward, loss, backward, step.

`evaluate`

Оценка без градиентов; опционально возвращает предсказания.

`train_model`

Обучение с ранней остановкой и сохранением лучшего состояния.

`run_one`

Один эксперимент в сетке: модель + оптимизатор + train_model.

`train_final_on_full`

Дообучение лучшей конфигурации на всех train-данных.

`build_grid`

Декартово произведение всех гиперпараметров + фильтр валидности.

`study_optimizers`

Сравнение оптимизаторов на одинаковой архитектуре.

`run_overfit_demo`

Маленькая выборка → переобучение → роль Dropout.

7.2. «Почему именно так»

- **Фиксируем seed перед каждой конфигурацией.** Иначе случайность инициализации мешает корректно сравнивать архитектуры.
- **AdaptiveAvgPool вместо ручного расчёта.** Не нужно знать размер тензора перед `Linear`; модель работает на любом входе.

- **Лучшая модель = max по `val_acc`**, а не по `test_acc`. Иначе оценка станет смещённой (мы фактически подберём модель под тест).
- **Финальное обучение на полном `train`**. После выбора архитектуры разумно использовать все доступные обучающие данные.
- **Ранняя остановка с возвратом к лучшему состоянию**. Защищает от лишних эпох и от ухода в переобучение.
- **Dropout перед `Linear`**. Случайно зануляет признаки во время обучения, заставляя модель не опираться на узкий набор детекторов.
- **`target_acc=1.0`, `patience=OVERFIT_EPOCHS`** в `overfit-demo`. Намеренно отключают раннюю остановку, чтобы переобучение успело развернуться.